

Integrating Ghidra Behavioral Similarity with the MISP Ecosystem

Thomas Caillet

CIRCL

14/04/2026, Hackaton.lu

Introduction

Introducing two open-source tools :

- 1 misp-ghidra
- 2 BsimVis

Correlating collection of binaries

At the fonctionnal level :

- Decompiled function signatures
- Behavioral feature vectors

To help analysts analyse collections of malware (or others)

<https://github.com/NationalSecurityAgency/ghidra>

Ghidra

- The open-source reverse engineering framework by NSA
- Widely used for static analysis :
 - ① Disassemble functions
 - ② Lifts to **P-code** intermediate representation (SLEIGH spec language)
 - ③ Decompile to **pseudo C** functions
- And debugging, version tracking
- Extensive plugin/scripting environment (Java & Python/PyGhidra)

Ghidra function similarity signatures

FID (Function ID):

- Finding exact match of known functions
- Opcode hashing = Arch / compiler **dependent**
- *e.g. finding statically linked functions in a stripped binary*

Disassemble function → Hash signature = 6A7C20F5BC6162A4

BSim (function Behavioral Similarity):

- Identify similar functions across different architectures and compilers
- P-code data/control flow = Arch / compiler **independent**
- *e.g. finding similar functions / different versions of variants of a malware*

Disassemble → P-code → Data/Control flow → Feature vector

BSim feature vector

Same as Machine Learning feature vectors

$$\mathbf{v}_{funcA} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ \vdots \\ 0 \\ 2 \\ \vdots \\ 0 \end{bmatrix} \quad \mathbf{v}_{funcB} = \begin{bmatrix} 1 \\ 0 \\ 1 \\ \vdots \\ 0 \\ 1 \\ \vdots \\ 1 \end{bmatrix}$$

Similarity

calculated using cosine similarity (with feature weighting) :

$$\text{sim}(funcA, funcB) = \cos(\mathbf{v}_{funcA}, \mathbf{v}_{funcB}) \in [0, 1]$$

<https://github.com/MISP/misp-ghidra>

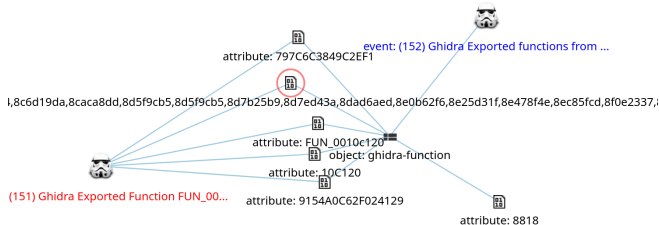
MISP-Ghidra

Ghidra scripts and python library to export Ghidra functions to MISP Threat Intelligence Platform:

- Export file metadata to MISP objects:
 - file objects, md5, ssdeep, PE/ELF/Mach-O headers
- Export functions as `ghidra-function` MISP objects:
 - C code, return types, address
 - FID and Bsim signatures

Correlation in MISP

- Function signatures match using the MISP correlation engine
- Across a collection of binaries
- Limited to exact matches, not similarities



Ghidra BSim databases limits

BSim databases

- Built for performant near neighbors search
- Not for storing similarities across a whole collection and perform analysis of these similarities

BSim usecases (1 to N binaries) :

- Finding similar known functions when analyzing one binary
- Version tracking
- Binary diffing : <https://github.com/clearbluejar/ghidriff>

Studying a collection of binaries

Examples of N to N analysis

Code reuse in a collection

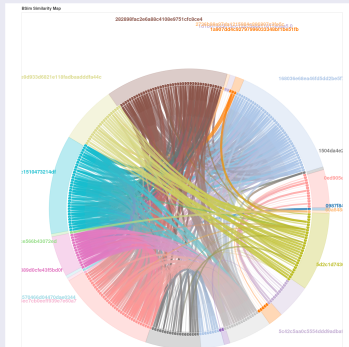


Figure 1: Similarity Chord diagram

Clustering binaries into families



Figure 2: Similarity cluster graph

<https://github.com/MISP/bsimvis>

BSimVis

Need for an alternative BSim database (Kvrocks db) :

- Store more context (P code, C code, features source mapping etc.)
- Store similarities for clustering and other subsequent analysis
- Allow for other similarity / ML algorithms

A tool for analyst to share binaries functions and explore a collection:

- Web interface and API
- upload, tag, search
- sharing results to MISP
- identify clusters, families, code reuse on many binaries

Similarity search

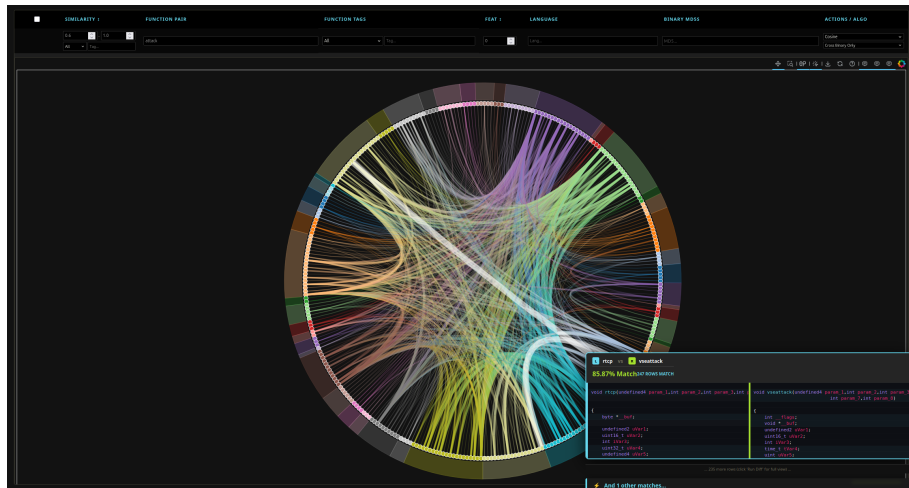


Figure 3: Similarity search/filtering across a collection of Mirai botnet variants

Diffing tool

80.1% match : SendHTTPCloudflare (MIPS) vs SendOVH_STORM (ARM)

```

22 if (param 4 == local 1c) {
23     return;
24 }
25 iVar2 = rand();
26 sprintf(acStack 220,
27     "%s /cdn-cgi/l/chk_captcha HTTP/1.1\r\nHost: %s\r\nUser-Agent: %s\r\nConnection: close\r\n\r\n",
28     param 1,param 2,(undefined4 *) (useragents + (iVar2 % 0x214 + 0)));
29     Var2 = fork();
30
31 if (Var3 != 0) break;
32 local 1c = local 1c + 1;
33 while (iVar1 = time((time_t *) 0) > local 30) {
34     local 20 = socket_connect(param 3,param 1);
35     if (local 20 != 0) {
36         n = strlen(acStack 220);
37         write(local 20,acStack 220, n);
38         read(local 20,&stack 221,1);
39         close(local 20);
40     }
41 }
42
43 // WARNING: Subroutine does not return */
44
45 while( true ) {
46     if (param 4 == local a20) {
47         return;
48     }
49     local 10 = rand();
50     local 10 = local 10 & 0x80000000;
51     if ((int)local 10 <= 0) {
52         local 10 = (local 10 - 1) & 0xffffffffL + 1;
53     }
54     sprintf(acStack a15,"GET %sStack 817,param 1,(undefined4 *) (useragents + local 10 * 4),
55     param2,param3,param4,param5,param6,param7,param8,param9,param10,param11,param12,param13,
56     param14,param15,param16,param17,param18,param19,param20,param21,param22,param23,param24,
57     param25,param26,param27,param28,param29,param30,param31,param32,param33,param34,param35,
58     param36,param37,param38,param39,param40,param41,param42,param43,param44,param45,param46,
59     param47,param48,param49,param50,param51,param52,param53,param54,param55,param56,param57,
60     param58,param59,param60,param61,param62,param63,param64,param65,param66,param67,param68,
61     param69,param70,param71,param72,param73,param74,param75,param76,param77,param78,param79,
62     param80,param81,param82,param83,param84,param85,param86,param87,param88,param89,param90,
63     param91,param92,param93,param94,param95,param96,param97,param98,param99,param100,
64     param101,param102,param103,param104,param105,param106,param107,param108,param109,
65     param110,param111,param112,param113,param114,param115,param116,param117,param118,
66     param119,param120,param121,param122,param123,param124,param125);
67     Var120 = fork();
68     if (Var120 != 0) break;
69     local a20 = local a20 + 1;
70 }
71 while (iVar127 = time((time_t *) 0) > local 30) {
72     fd = socket_connect(param 3,param 2);
73     if (fd != 0) {
74         n = strlen(acStack a15);
75         write(fd,acStack a15,n);
76         read(fd,&stack 018,1);
77         close(fd);
78     }
79 }
80
81 // WARNING: Subroutine does not return */
82

```

Figure 4: Diffing view based on Bsim feature vector and mapping to C code

Mirai/Gafgyt botnet functions that fork the process, create sockets in a loop and send randomized payloads, but for different methods/targets

Visualizing individual feature context

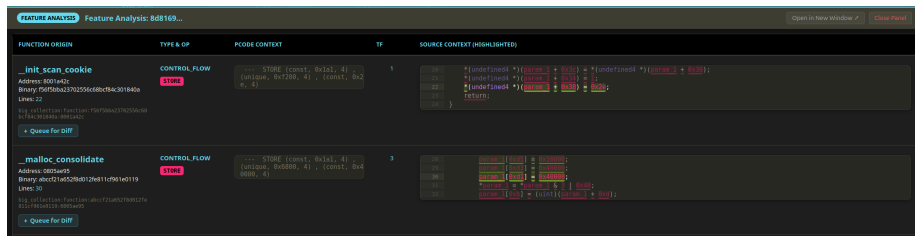


Figure 5: Reimplementation of the BSim Feature Visualiser plugin across multiple binaries

Visualizing a feature across the collection:

- Its different representations
- Functions involved
- C code / P-code context

Other features

- Binaries are grouped in collections
- Job queue system
- Tagging and searching at the binary, function and similarity level

e.g. Analyst tags a malicious function/binary, searches similar functions, bookmarks the newly found similarities for further analysis, potentially identifying a family of malware in the dataset

Use cases

Identify clusters, families, code reuse on many binaries :

- Malware families, variants and evolution
- Firmware / software versions, compiled for different platform

Conclusion

What's next ?

- Clustering algorithms (K-means, DBSCAN, etc.)
- Machine learning algorithms (tf idf, Dimension reduction, PCA, etc.)
- Exclusion filters for benign functions (libc libraries, etc.)
- Binary level diffing, clustering
- Integratation with MISP ecosystem, VirusTotal and other databases

Hackaton team to work on it

- Web interface
- New features

Clustering Malware at Scale: A First Full-Benchmark Study, 2025

<https://arxiv.org/abs/2511.23198>

Is Function Similarity Over-Engineered? Building a Benchmark, 2024

<https://arxiv.org/pdf/2410.22677v1>

MutantX-S: Scalable Malware Clustering Based on Static Features, 2013

<https://www.usenix.org/conference/atc13/technical-sessions/presentation/hu>